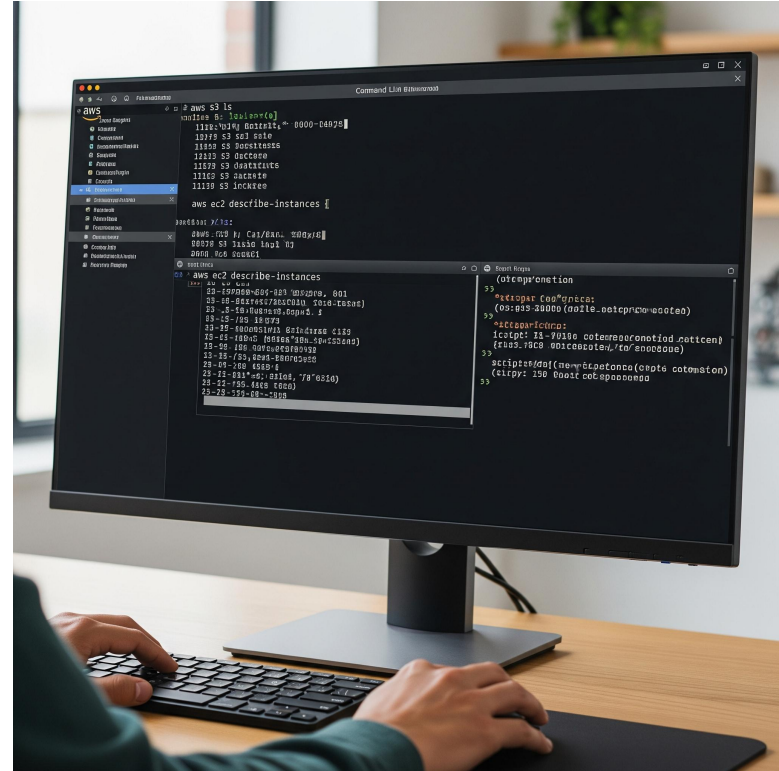


AWS CLI

AWS CLI

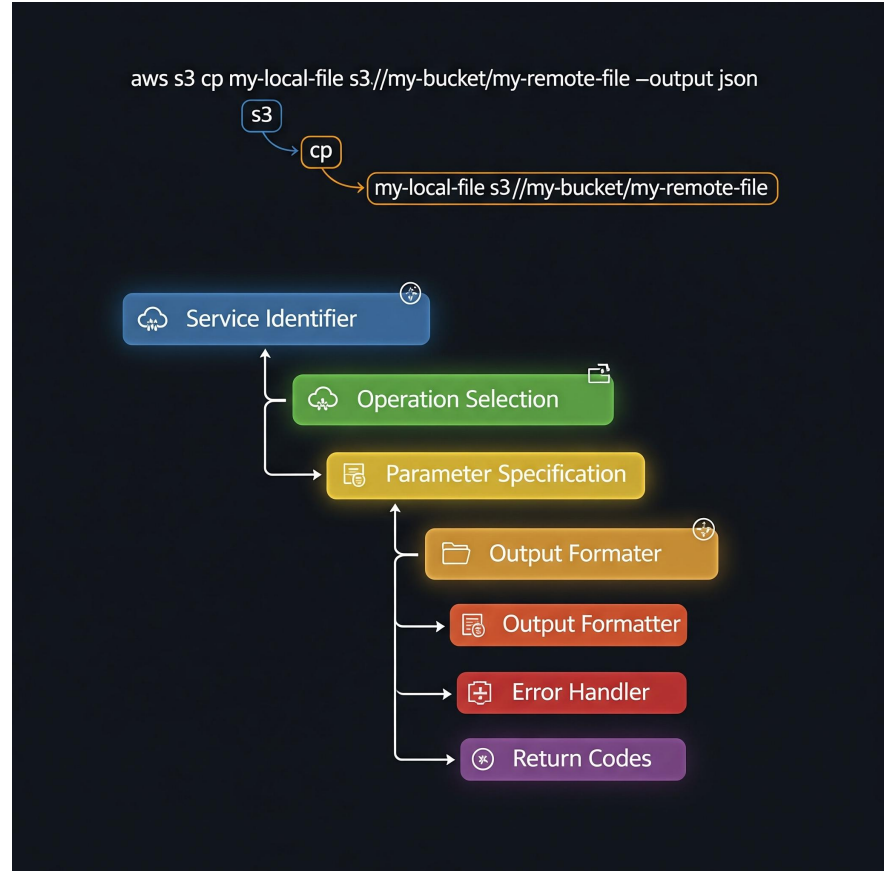
- ❖ AWS Command Line Interface (CLI) is a unified tool to manage AWS services from the command line.
- ❖ It provides direct access to AWS services' public APIs and enables automation through scripts.



Command Line Structure

Command Format:

- ❖ Service Identifier
- ❖ Operation Selection
- ❖ Parameter Specification
- ❖ Output Formatter
- ❖ Error Handler
- ❖ Return Codes



Parameter Types

- ❖ Required Parameter
- ❖ Optional Parameter
- ❖ Positional Parameter
- ❖ Named Parameter
- ❖ JSON-Input Parameter
- ❖ File-based Parameter

AWS CLI Parametrs

Required Parameter

```
aws ec2 run-instances --image-id  
ami-0abcdef1234567890
```

Positional Parameter

```
aws s3 ls --recursive
```

Named Parameter

```
aws s3api get-object {  
  -bucket my-filet-bucket --key  
  key my-file.txt/  
}
```

JSON-Input Parameter

```
aws iam create-user  
myuser '{"Key": "Project", "Value": "Alpha"}'
```

Optional Parameter

```
aws s3 ls --recursive
```

```
aws s3api get-object  
-bucket my-bucket --s3//my-bucket/
```

JSON-Input Parameter

```
aws iam create-user --user-name  
myugs "[{"Key": "Project", "Value": "Alpha"}]"
```

File-based Parameter

```
aws ec2 run-instances --cli-nput json  
file://instance-config.json
```


Command Processing Step

Step:1 Input Handling

Step:2 Parameter Validation

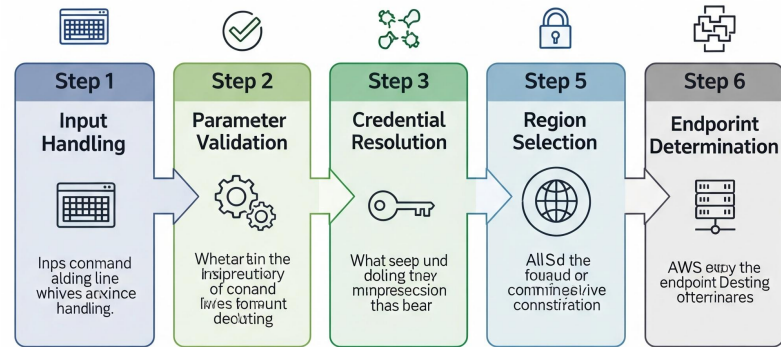
Step:3 Input Transformation

Step:4 Credential Resolution

Step:5 Region Selection

Step:6 Endpoint determination

AWS CLI command Processing Steps



Command Processing Output Steps

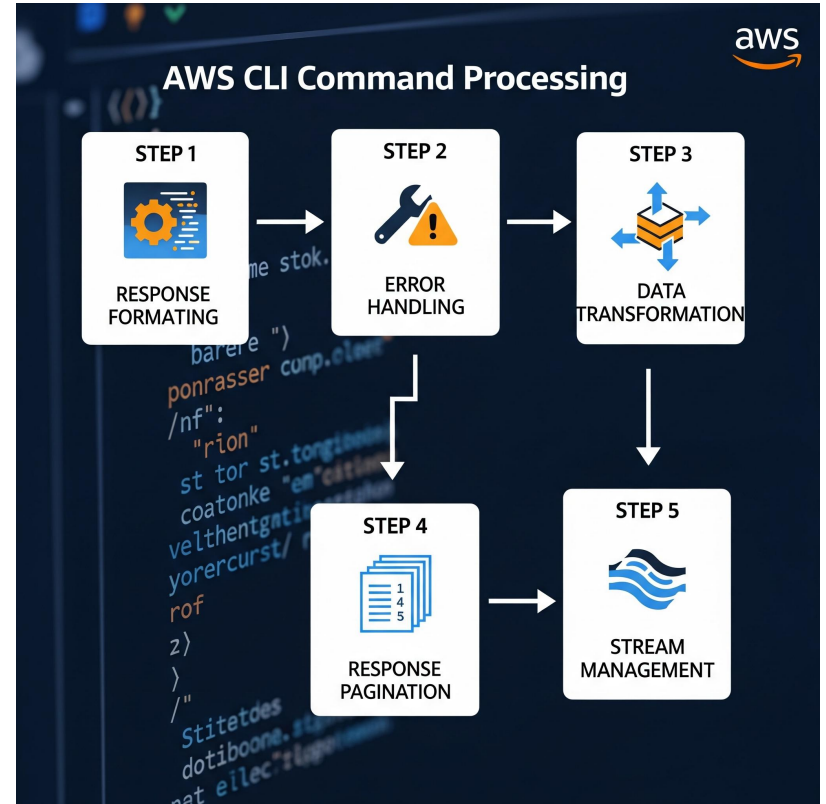
Step:1 Response Formatting

Step:2 Error Handling

Step:3 Data Transformation

Step:4 Response Pagination

Step:5 Stream Management



Configuration Management

Profile Configuration

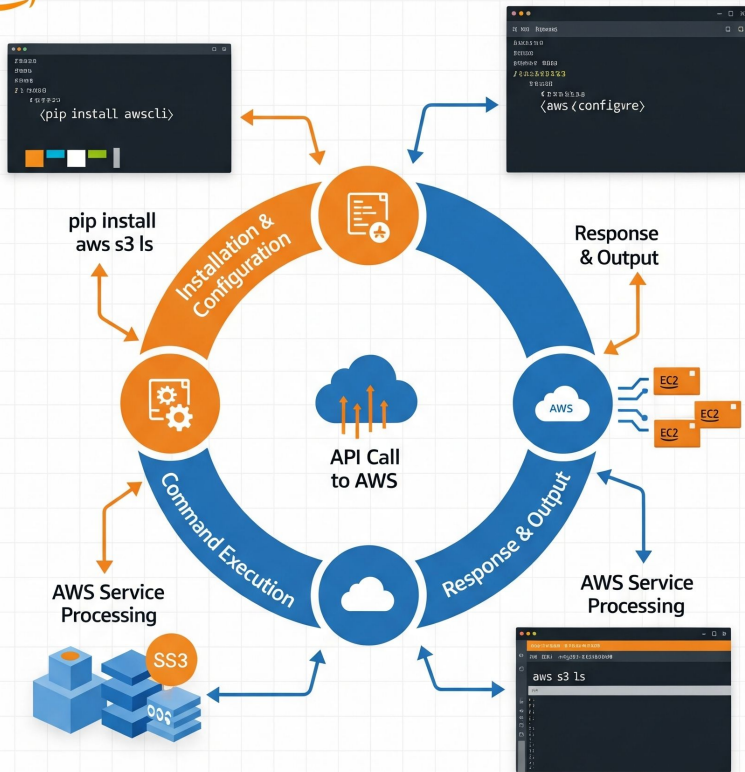
Credentials Config

- ❖ Access key
- ❖ Secret Key Storage
- ❖ Session Token
- ❖ Role Assumption
- ❖ External credential Provider
- ❖ Credential Rotation

Regional Settings

- ❖ Default Region Configuration
- ❖ Multiple Region Support
- ❖ Endpoint Customization
- ❖ Service Endpoint
- ❖ Global Service handling
- ❖ Regional Service Availability

aws CLI



Credenttian & Regional Settings

Credential Configuration



API Keys



Usernames & Passwords



OAuth Tokens



SSH Keys

- Your Redstvard -API Curen

Regional Settings



Data Center Location



Language & Locale



Time Zone



Currency

Environment Variables-System Configuration

- ❖ **AWS_ACCESS_KEY_ID:** e AWS access key identifier used for authentication,
- ❖ **AWS_SECRET_ACCESS_KEY:** Securely stores the secret access key associated with the access key ID,
- AWS_SESSION_TOKEN:** Holds temporary security credentials for assumed roles or federated users,
- WS_DEFAULT_REGION:** Specifies the default AWS region for CLI operations
- ❖ **AWS_DEFAULT_OUTPUT:** Defines the default output format (json, text, or table)



Environment Variable: Proxy Settings

HTTP_PROXY: Specifies the HTTP proxy server configuration for CLI requests,

HTTPS_PROXY: Defines the HTTPS proxy server for secure CLI connections,

AWS_CA_BUNDLE: Points to a custom certificate authority (CA) bundle file,.

NO_PROXY: Lists domains or IP addresses that should bypass proxy settings, optimizing performance for internal resources or directly accessible AWS endpoints.

AWS_DEFAULT_PROFILE: Identifies the default credential profile to use

AWS_CONFIG_FILE: Specifies the location of the AWS CLI configuration file,



Case:1 Managing Instances using CLI

Instance Management commands-1

- Instance Management:

Start and stop instances

```
aws ec2 start-instances --instance-ids  
i-1234567890abcdef0  
aws ec2 stop-instances --instance-ids  
i-1234567890abcdef0
```

Modify instance attributes

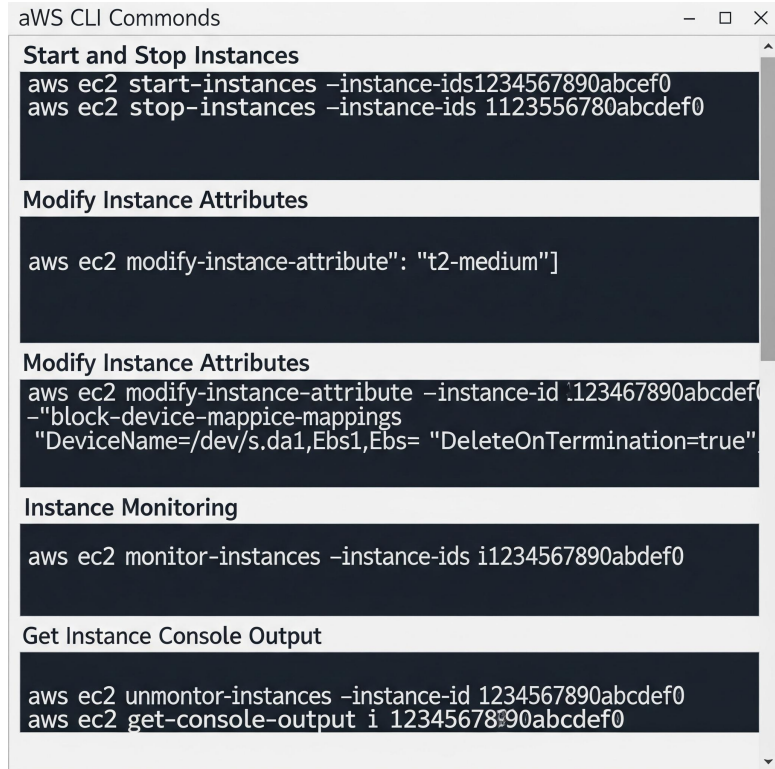
```
aws ec2 modify-instance-attribute \  
--instance-id i-1234567890abcdef0 \  
--instance-type t3.medium
```

Instance monitoring

```
aws ec2 monitor-instances --instance-ids  
i-1234567890abcdef0  
aws ec2 unmonitor-instances --instance-ids  
i-1234567890abcdef0
```

Get instance console output

```
aws ec2 get-console-output --instance-id  
i-1234567890abcdef0
```



```
aWS CLI Commonds  
  
Start and Stop Instances  
aws ec2 start-instances --instance-ids i-1234567890abcdef0  
aws ec2 stop-instances --instance-ids i-1234567890abcdef0  
  
Modify Instance Attributes  
  
aws ec2 modify-instance-attribute --instance-id i-1234567890abcdef0 --attribute "t2-medium"  
  
Modify Instance Attributes  
aws ec2 modify-instance-attribute --instance-id i-1234567890abcdef0 --attribute "block-device-mappings" --value [{"DeviceName": "/dev/sda1", "Ebs": {"DeleteOnTermination": true}}]  
  
Instance Monitoring  
aws ec2 monitor-instances --instance-ids i-1234567890abcdef0  
  
Get Instance Console Output  
aws ec2 unmonitor-instances --instance-id i-1234567890abcdef0  
aws ec2 get-console-output --instance-id i-1234567890abcdef0
```

Instance Management commands -2

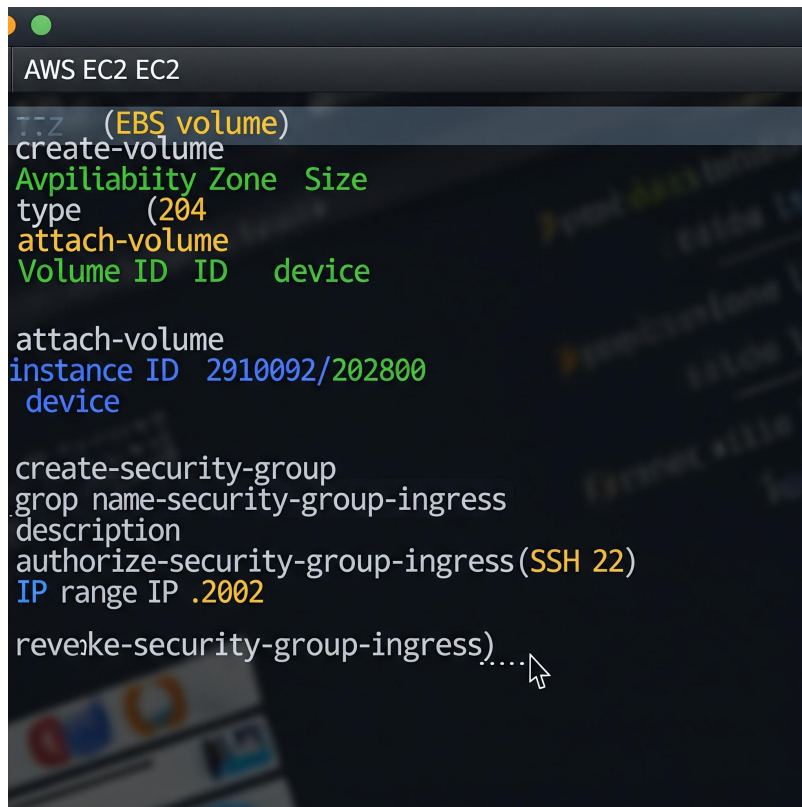
Create and attach EBS volumes

```
aws ec2 create-volume \  
  --volume-type gp3 \  
  --size 100 \  
  --availability-zone us-west-2a
```

```
aws ec2 attach-volume \  
  --volume-id vol-1234567890abcdef0 \  
  --instance-id i-1234567890abcdef0 \  
  --device /dev/sdf
```

Manage security groups

```
aws ec2 authorize-security-group-ingress \  
  --group-id sg-1234567890abcdef0 \  
  --protocol tcp \  
  --port 80 \  
  --cidr 0.0.0.0/0
```



```
AWS EC2 EC2  
  
... (EBS volume)  
create-volume  
Availability Zone Size  
type (gp3)  
attach-volume  
Volume ID ID device  
  
attach-volume  
instance ID i-1234567890abcdef0  
device /dev/sdf  
  
create-security-group  
group name security-group-ingress  
description  
authorize-security-group-ingress (SSH 22)  
IP range IP 0.0.0.0/0  
revoke-security-group-ingress)....
```

Case 2 : S3 Bucket Management

Bucket Management Commands-1

Create bucket with settings

```
aws s3api create-bucket \
  --bucket my-new-bucket \
  --region us-west-2 \
  --create-bucket-configuration
LocationConstraint=us-west-2 \
  --object-ownership BucketOwnerPreferred
```

Enable versioning

```
aws s3api put-bucket-versioning \
  --bucket my-bucket \
  --versioning-configuration Status=Enabled
```

Configure lifecycle rules

```
aws s3api put-bucket-lifecycle-configuration \
  --bucket my-bucket \
  --lifecycle-configuration
file:///lifecycle-rules.json
```

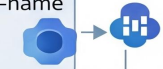
Create Bucket

```
aws s3api create-bucket --bucket my-unique-bucket-name --region us-east-1 \
  --create-bucket-configuration LocationConstraint=us-east --acl=private}
```



Enable Versioning

```
aws s3api put-bucket-versioning --bucket-unique-bucket-name \
  --versioning-configuration Status=Enabled
```



Configure Lifecycle Rules

```
aws s3api put-bucket-lifecycle-configuration --bucket my-configuration file:///lifecycle-config.json
```

```
{
  "rules": [
    {
      "id": "lifecycle-Glacier-(Glacier after 30 days)",
      "filter": {
        "prefix": "lifecycle-Glacier after-config.json"
      },
      "transitions": [
        {
          "days": 30,
          "storage_class": "GLACIER"
        }
      ]
    }
  ]
}
```

Lifecycle.json

```
transitioning
objects
shifter objects
after 30
expiring
```

Bucket Management Command 2

Single file upload with metadata

```
aws s3 cp myfile.txt s3://my-bucket/myfile.txt \
--metadata '{"key1":"value1","key2":"value2"}' \
--storage-class STANDARD_IA \
--sse aws:kms \
--sse-kms-key-id
```

arn:aws:kms:region:account:key/key-id

Directory sync with filters

```
aws s3 sync . s3://my-bucket/backup \
--exclude "*.tmp" \
--include "*.jpg" \
--delete \
--acl bucket-owner-full-control
```

Multipart upload for large files

```
aws s3 cp largefile.zip s3://my-bucket/ \
--multipart-threshold 100MB \
--part-size 100MB
```

Data data Transfer

single file



```
aws s3 cp myfile-bucket/my-file.txt
--metadata '"key1" "value1", "value1,,key2:"value2"'
--storage-class STANDARD_IA
--sseaws:kms --sse-kms:km:skimen:region:accountk:key-id
s3.13
```

```
aws s3 sync . s3://my-buckeup --exclude "t.jpg" --i*.jpg" ".jpg"
--delete --acl bucketownerfull-control
```

```
aws s3 cp largefile.zip s3://my-bucket/
--multipart-threshold 100MB --part-size 100MB
```

```
aws s3 cp largfile.dat/s3/mpy-100MB --part-size 100MB
```

```
aws s3 cp bigfile.dat/ s3//xpected-size0000 --quiet --progress
```

IAM CLI Command

User Management

User Operations:

Create new IAM user

```
aws iam create-user \  
  --user-name new-employee \  
  --tags Key=Department,Value=Engineering  
Key=Role,Value=Developer
```

Add user to groups

```
aws iam add-user-to-group \  
  --user-name new-employee \  
  --group-name developers
```

Create login profile

```
aws iam create-login-profile \  
  --user-name new-employee \  
  --password 'InitialPassword123!' \  
  --password-reset-required
```

Create new IAM user



Navigate to IAM console
Users > Add user
Set user name and access type



Add user to groups



Select user
Add user to group
Choose existing or create new group



Create login profile



Select user
Create login profile
Set and confirm password



Group Management

Create new group

```
aws iam create-group \  
  --group-name developers
```

Attach policy to group

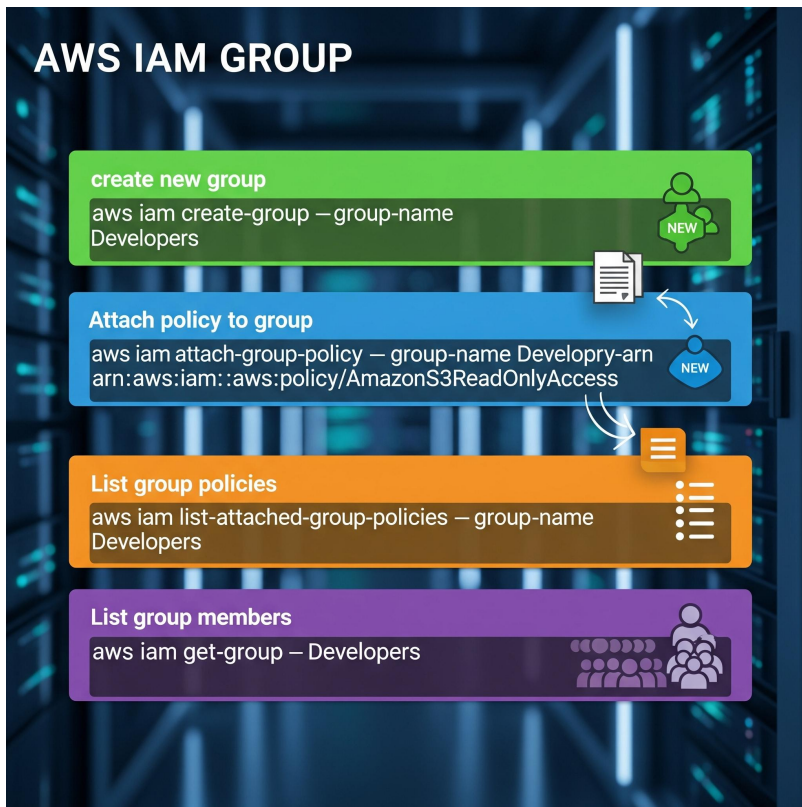
```
aws iam attach-group-policy \  
  --group-name developers \  
  --policy-arn  
arn:aws:iam::aws:policy/PowerUserAccess
```

List group policies

```
aws iam list-attached-group-policies \  
  --group-name developers
```

List group members

```
aws iam get-group \  
  --group-name developers
```



Policy Management

Create custom policy

```
aws iam create-policy \  
  --policy-name custom-s3-policy \  
  --policy-document '{  
    "Version": "2012-10-17",  
    "Statement": [{  
      "Effect": "Allow",  
      "Action": ["s3:GetObject",  
"s3:PutObject"],  
      "Resource": "arn:aws:s3:::my-bucket/*"  
    }]  
  }'
```

List policies by scope

```
aws iam list-policies \  
  --scope Local \  
  --only-attached
```

Create New Group

Group Name

Group Name

Attach Policy to Group [Create Group](#)

List Group Policies ▾

Policy	ARN	Attached Date
Polity	ARN	17.000
Polity	ARN	25.000

List Group Members

Policy Name	ARN	ARN	Added Date
User Name	ARN	13.00	1350000
User Name	ARN	23.00	1650000
User Name	ARN	23.00	1250000

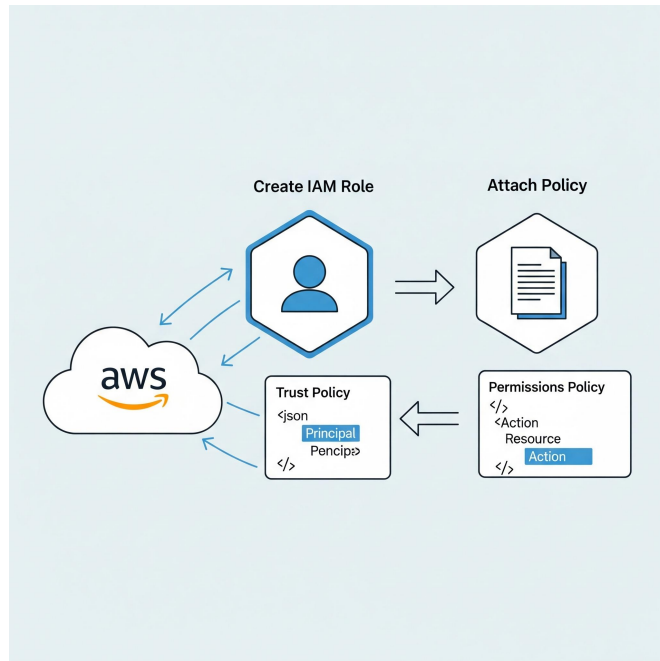
Role Management

Create role with trust policy

```
aws iam create-role \  
  --role-name lambda-execution \  
  --assume-role-policy-document '{  
    "Version": "2012-10-17",  
    "Statement": [{  
      "Effect": "Allow",  
      "Principal": {  
        "Service": "lambda.amazonaws.com"  
      },  
      "Action": "sts:AssumeRole"  
    }]  
  }'
```

Attach policy to role

```
aws iam attach-role-policy \  
  --role-name lambda-execution \  
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
```



AWS CLI Cloudwatch

Monitoring Operations

Put custom metric data

```
aws cloudwatch put-metric-data \  
  --namespace "CustomMetrics" \  
  --metric-name "PageViewCount" \  
  --value 100 \  
  --dimensions Name=SiteName,Value=example.com \  
  --timestamp $(date -u +"%Y-%m-%dT%H:%M:%SZ")
```

Get metric statistics

```
aws cloudwatch get-metric-statistics \  
  --namespace AWS/EC2 \  
  --metric-name CPUUtilization \  
  --dimensions
```

```
Name=InstanceId,Value=i-1234567890abcdef0 \  
  --start-time "2025-09-10T00:00:00" \  
  --end-time "2025-09-11T00:00:00" \  
  --period 3600 \  
  --statistics Average Maximum
```

aws cloudwatch put-metric-data

```
--aws cloudwatch put-metric-data  
[[{"MetricName" "CPUUtilization"  
  Dimensions" InstanceId" "Value"  
  Timestamp i-0abcdef1234567890"}]]  
Value; 72.1, 72.1  
Unit: Percent"]  
[[{"MetricName" "MemoryUsage"  
  Dimensions" InstanceId" "Value"  
  Timestamp i-0abcdef1234567890"}]]  
Timestamp, Value, 10:01:00Z"]  
Unit: Percent"]  
--namespace "MyApplication/Servers"
```

aws cloudwatch get-metric-statistics

```
aws cloudwatch get-metric-statistics  
--namespace "Myapplication/Servers"  
--metric-name "CPUUtilization"  
--Dimensions Name=InstanceId,Value=  
i-0abcdef1234567890  
--start-Time "2023-10-27T09:00:00Z"  
--end-time "2023-10-27T10:00:00Z"  
--Period 300  
--statistics Average--Percent
```

Alarm Configuration

Create metric alarm

```
aws cloudwatch put-metric-alarm \  
  --alarm-name high-cpu-utilization \  
  --alarm-description "CPU usage exceeds 80%" \  
  --metric-name CPUUtilization \  
  --namespace AWS/EC2 \  
  --dimensions
```

```
Name=InstanceId,Value=i-1234567890abcdef0 \  
  --period 300 \  
  --threshold 80 \  
  --comparison-operator GreaterThanThreshold \  
  --evaluation-periods 2 \  
  --statistic Average \  
  --alarm-actions arn:aws:sns:region:account-id:topic-name
```

List and describe alarms

```
aws cloudwatch describe-alarms \  
  --alarm-names high-cpu-utilization \  
  --state-value ALARM
```

aws CloudWatch

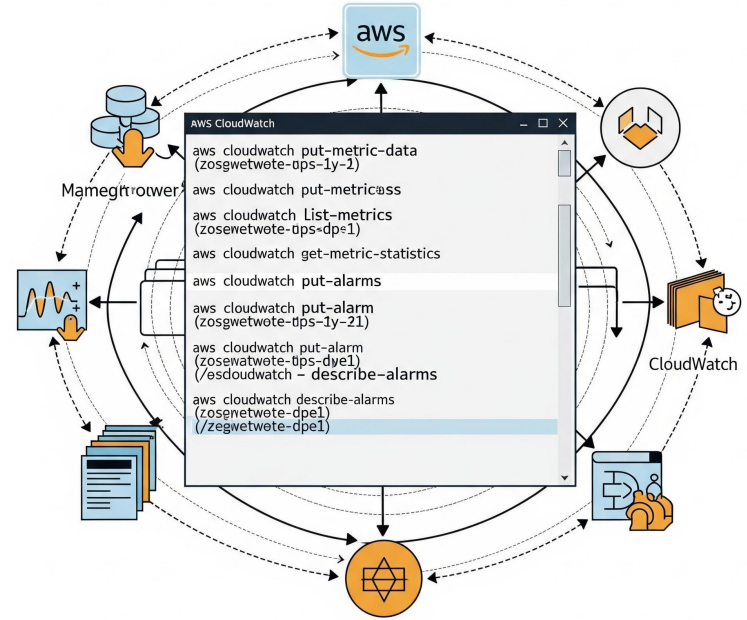
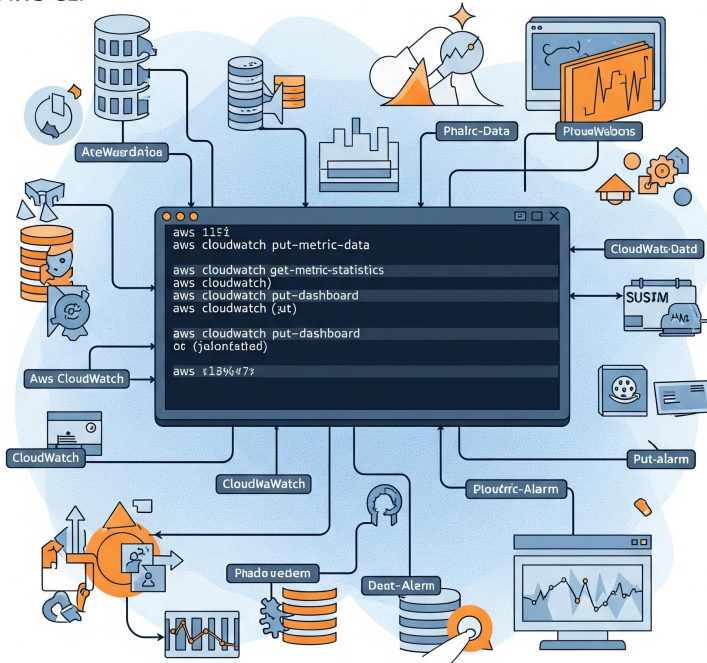
```
aws cloudwatch put-metric-alarm  
[--alarm-name  
[--metric-name  
[--namespace  
[--namespace  
[--statistic  
[--period  
[--period]  
  
[--threshold]  
[--comparison-operator]  
[--evaluation-periods]  
  
aws cloudwatch describe-alarms]  
[--alarm-names [6b      [--state-value]
```

AWS CLI Cloudwatch management



aws CloudWatch Management

AWS CLI



Bash Scripting

Script Integration for Automation

- ❖ Bash Scripting
- ❖ PowerShell
- ❖ Programming Languages



Bash Script Basics

- ❖ AWS CLI Integration
- ❖ Conditional Logic
- ❖ Looping Constructs
- ❖ Error Handling
- ❖ Variable Management
- ❖ Functions
- ❖ Input Output



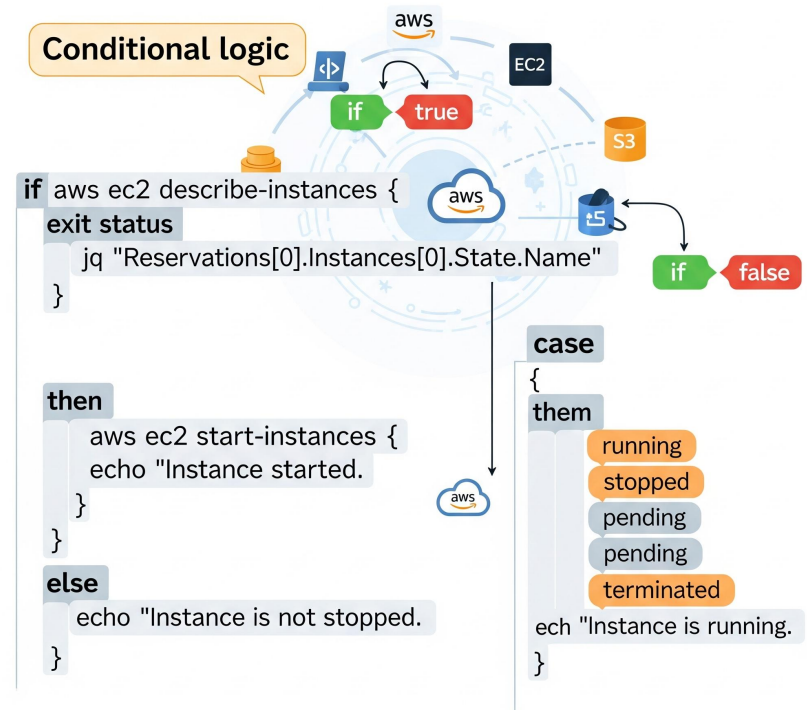
AWS CLI Integration

- ❖ Directly execute AWS CLI commands within Bash scripts.
- ❖ Capture command output for further processing using shell variables.
- ❖ Utilize `--query` and `--output` flags for precise data extraction.



Conditional Logic

- Implement `if/else` statements based on AWS CLI command success or failure (using `$?` or `if aws ...`).
- Check for specific values in JSON output using `jq` and conditional expressions.
- Use `case` statements for different scenarios based on AWS resource states.



Looping Constructs

- Iterate through lists of AWS resource IDs (e.g., instance IDs, bucket names) using `for` loops.
- Process data from AWS CLI `describe` or `list` commands by piping output to `while read` loops.
- Automate repetitive tasks like creating multiple resources or updating configurations.

for For Loop: Iterating AWS Resource IDs

`for`

```
i-0abcdef1234567890  
i-0abcdef1234567890  
i-1abcdef1234567890  
i-0abcdef1234567890  
i-0abcdef1234567890  
i-2abcdef1234567890  
i-1abcdef1234567890  
i-0abcdef1234567890
```

`aws ec2 stop-instances
--instance-ids $instance_id`



`aws ec2--instances
--instance-ids $instance_id`



while While Read Loop: Processing AWS CLI Output
`aws ec2 describe-instances --query 'Reservations[[]].InstanceId'`

```
while read instance_id; do ... done
```



```
aws ec2 monitor-instances --instance-ids $instance_id'
```



for Automating Repetitive Tasks
`for`

```
us-east-1  
us-east-2  
us-west-2
```

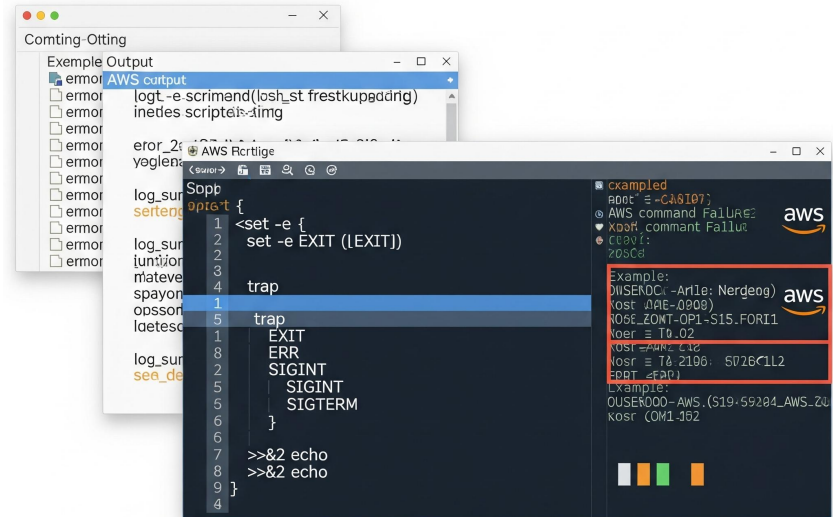


```
aws s3api create-Bucket --bucket my-bucket-$region $region  
aws ec2 run-instances ... --region $region
```

Error Handling

- Use `set -e` to exit the script immediately if a command fails.
- Implement `trap` commands to catch signals and perform cleanup actions.
- Print informative error messages to `stderr` using `>&2 echo`.
- Log command output and errors to files for debugging and auditing.

Error Handling



The image displays three overlapping terminal windows. The top window, titled 'Comting-Otting', shows a script snippet with error handling logic. The middle window, titled 'AWS output', shows a script snippet with error handling logic. The bottom window, titled 'exampled', shows the output of an AWS CLI command, with a red box highlighting the error message.

```
Comting-Otting
Example Output
error AWS output
logt -e scrimand(losht_st frestkupaddrig)
inides scriptel&ding
error_2f AWS Rcttge
veglen
log_sur Stpb
seteng optg
1 {
2 <set -e {
3 set -e EXIT (LEXIT))
4
5 trap
6
7 EXIT
8 ERR
9 SIGINT
10 SIGINT
11 SIGTERM
12 }
13
14 >>&2 echo
15 >>&2 echo
16 }
```

```
exampled
root@ ~ # aws s3 cp s3://aws-logs-1-uk-east-1.amazonaws.com/2018-01-01/2018-01-01_01-01-01.log.gz s3://aws-logs-1-uk-east-1.amazonaws.com/2018-01-01/2018-01-01_01-01-01.log.gz
aws s3 cp s3://aws-logs-1-uk-east-1.amazonaws.com/2018-01-01/2018-01-01_01-01-01.log.gz s3://aws-logs-1-uk-east-1.amazonaws.com/2018-01-01/2018-01-01_01-01-01.log.gz: command failed: exit status 1
Example:
DUSEKDCI -Arlle: Nerdgeog)
Host: APE-0008
ROSE_ZOMT-OP1-S15.FOR11
Host: T9_02
Host: APE-012
Host: T6_2106_5026C112
Host: APE-012
Example:
DUSEKDCI -Arlle: Nerdgeog)
Host: APE-0008
ROSE_ZOMT-OP1-S15.FOR11
Host: T9_02
Host: APE-012
Host: T6_2106_5026C112
Host: APE-012
```

Variable Management

- Store sensitive information (e.g., access keys, secret keys) securely using environment variables or AWS Secrets Manager, avoiding hardcoding in scripts.
- Use shell variables to store dynamic data like resource IDs, names, and configuration parameters.
- Pass variables between functions or scripts as arguments.



Storing Sensitive Information

Corrupted:
#AWS_ACCESS_KEY_ID=\$KMS
\$(aws secretsmanager get-secret-value --secret-id MyAccessKeyquery
SecretString-output text)

Using Shell Variables

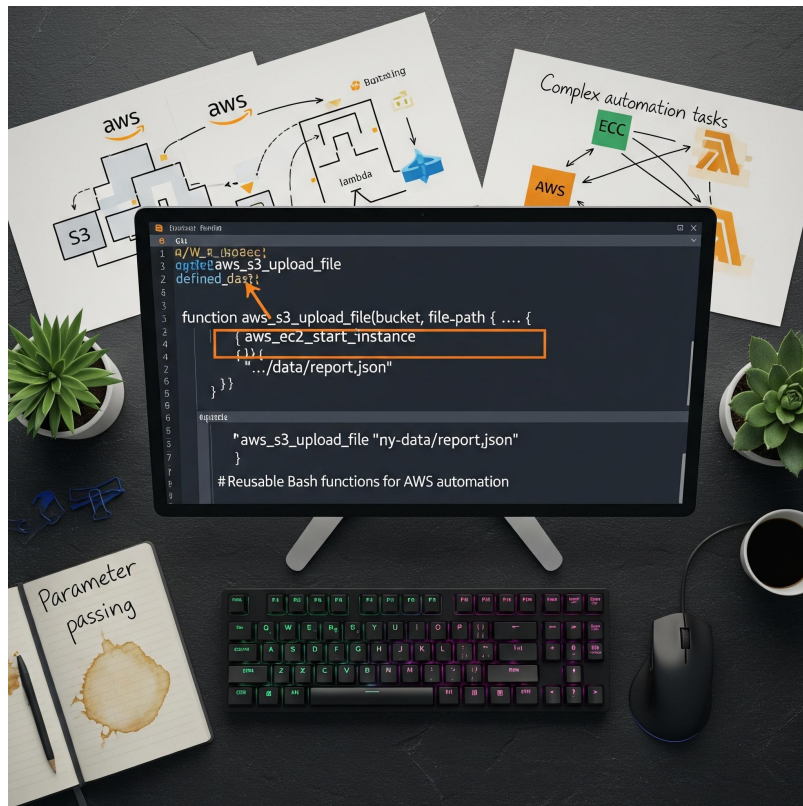
User variables
AWS_REGION=us-east-1
EC2 instance
INSTANCE_TYPE=t2.micro"

Passing Variables Between Functions

function:
deploy_instance() { ... {
\$1
\$2
deploy_instance \$INSTANCE_TYPE "AMIbcdef1234567890"

Functions

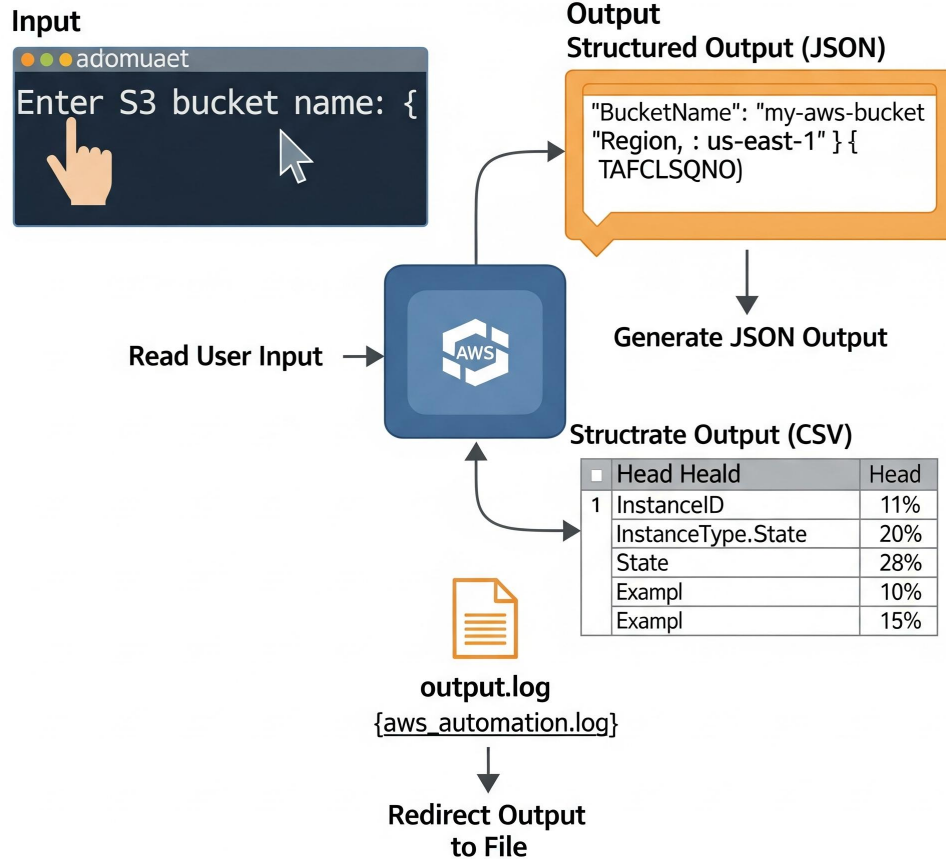
- Organize complex automation tasks into reusable Bash functions.
- Define functions to encapsulate specific AWS operations, improving readability and maintainability.
- Pass parameters to functions for flexibility and reusability.



Input Output

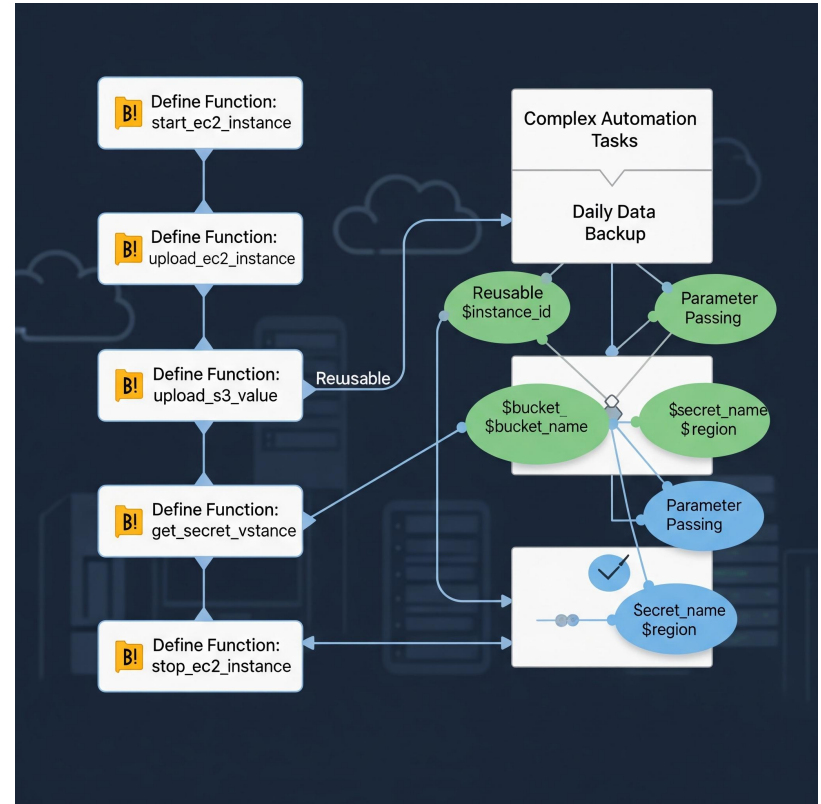
- Read user input using `read` for interactive scripts.
- Generate structured output (e.g., JSON, CSV) from AWS CLI data.
- Redirect output to files for logging or to other commands for further processing.

Bash Scripting for



Scheduling and Execution

- Schedule Bash scripts using `cron` for periodic automation.
- Integrate scripts with CI/CD pipelines (e.g., Jenkins, GitLab CI) for automated deployments and operations.
- Execute scripts remotely on EC2 instances or other servers



Powershell for AWS Automation

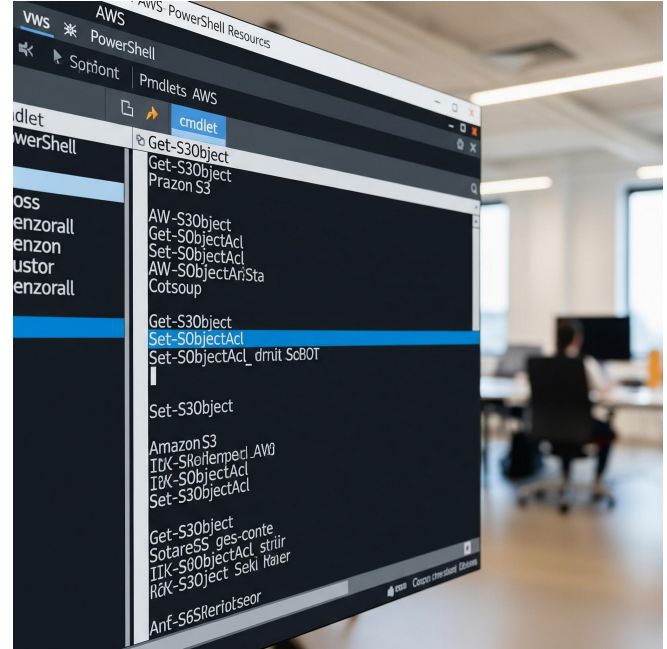
PowerShell Automation

- ❖ Utilize AWS PowerShell module for efficient management.
- ❖ Automate AWS tasks using PowerShell cmdlets.
- ❖ Integrate PowerShell scripts into existing workflows.
- ❖ Leverage PowerShell for robust cloud operations.



PowerShell Cmdlets

- ❖ Manage AWS services with PowerShell cmdlets
- ❖ Use cmdlets for efficient cloud resource control
- ❖ Automate AWS tasks using PowerShell scripts



Key PowerShell Cmdlets



01 Efficiently manage AWS services with PowerShell cmdlets.

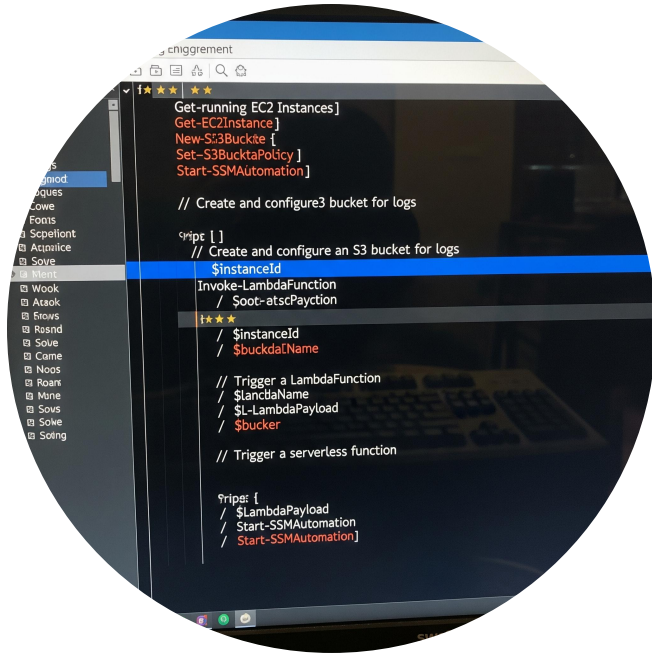
03 Automate diverse AWS tasks using powerful cmdlets.

02 Integrate cmdlets for robust cloud operational workflows.

04 Streamline cloud resource control via PowerShell.

PowerShell Script Example

- Create an example PowerShell script for AWS.
- Show how to manage AWS services effectively.
- Illustrate efficient cloud resource control.
- Demonstrate script integration for automation.



Powershell Examples:Listing Buckets

```
# Install AWS Tools for PowerShell if you haven't already  
# Install-Module -Name AWSPowerShell.NetCore -Scope CurrentUser
```

```
# List all S3 buckets in your default region
```

```
Get-S3Bucket
```

```
# List S3 buckets in a specific region
```

```
Get-S3Bucket -Region us-east-1
```

Powershell: Creating Instance

Define parameters for the EC2 instance

```
$amild = "ami-0abcdef1234567890" # Replace with a valid AMI ID  
$instanceType = "t2.micro"  
$keyPairName = "my-key-pair" # Replace with an existing key pair name  
$securityGroupIds = @("sg-0123456789abcdef0") # Replace with your security group ID
```

Create a new EC2 instance

```
New-EC2Instance -ImageId $amild -InstanceType $instanceType -KeyName $keyPairName -SecurityGroupIds  
$securityGroupIds -MinCount 1 -MaxCount 1
```

Tag the instance

```
$instanceld = (Get-EC2Instance -Filter  
@{Name="instance-state-name";Values="pending"}).Instances.InstanceId  
New-EC2Tag -ResourceId $instanceld -Tags @{Key="Name";Value="MyTestInstance"}
```


PowerShell Error Handling

- Utilize try-catch blocks for robust error management.
- Implement error handling for AWS PowerShell cmdlets.
- Log errors to files for debugging and auditing.
- Use error variables to inspect detailed error information.



Programming Languages

Boto3 for AWS Python



- Utilize Boto3 for AWS service interaction.
- Automate AWS tasks with Python scripts.
- Manage cloud resources efficiently using Boto3.
- Integrate Python for robust cloud operations.

Python Integration Example

```
def list_instances_by_tag(tag_key, tag_value):
    try:
        ec2 = boto3.resource('ec2')
        instances = ec2.instances.filter(
            Filters=[{
                'Name': f'tag:{tag_key}',
                'Values': [tag_value]
            }]
        )
        return [{'id': i.id, 'state': i.state['Name']} for i
in instances]
    except ClientError as e:
        print(f"Error listing instances: {e}")
        return []
```

Python Integration: Resource Management

```
import boto3
from concurrent.futures import ThreadPoolExecutor

def get_bucket_size(bucket_name):
    try:
        s3 = boto3.client('s3')
        paginator = s3.get_paginator('list_objects_v2')
        total_size = 0

        for page in paginator.paginate(Bucket=bucket_name):
            if 'Contents' in page:
                total_size += sum(obj['Size'] for obj in page['Contents'])

        return {
            'bucket': bucket_name,
            'size_bytes': total_size,
            'size_gb': total_size / (1024**3)
        }
    except ClientError as e:
        return {'bucket': bucket_name, 'error': str(e)}
```

Automation Tools

- ❖ Node.js support
- ❖ Java integration
- ❖ NET capabilities
- ❖ Ruby implementation
- ❖ Go programming
- ❖ PHP integration



Integration Capabilities

- ❖ Service access
- ❖ Resource management
- ❖ File transfer
- ❖ Command history
- ❖ Tab completion
- ❖ Tool updates



Knowledge Checks

Knowledge Check-1

Which of the following is the correct structure for an AWS CLI command?

- a) aws <operation> <service> [parameters]
- b) aws <service> <operation> [parameters]
- c) aws [parameters] <service> <operation>
- d) aws <operation> [parameters] <service>

Knowledge Checks-2

Which parameter type is mandatory for a command to execute successfully?

- a) Optional Parameter
- b) Positional Parameter
- c) Required Parameter
- d) Named Parameter

Knowledge Check-3

In the AWS CLI command processing steps, what happens during "Credential Resolution"?

- a) The command is broken down into its components.
- b) Raw input parameters are converted to the appropriate format.
- c) AWS credentials are located and validated.
- d) The specific AWS service endpoint URL is resolved.

Knowledge Check -4

When managing S3 buckets using the AWS CLI, which command would you use to enable versioning on a bucket?

- a) `aws s3api create-bucket-versioning`
- b) `aws s3api put-bucket-versioning`
- c) `aws s3 enable-versioning`
- d) `aws s3api update-bucket-versioning`

Knowledge Check-5

What is the primary benefit of using PowerShell cmdlets for AWS automation compared to direct API calls?

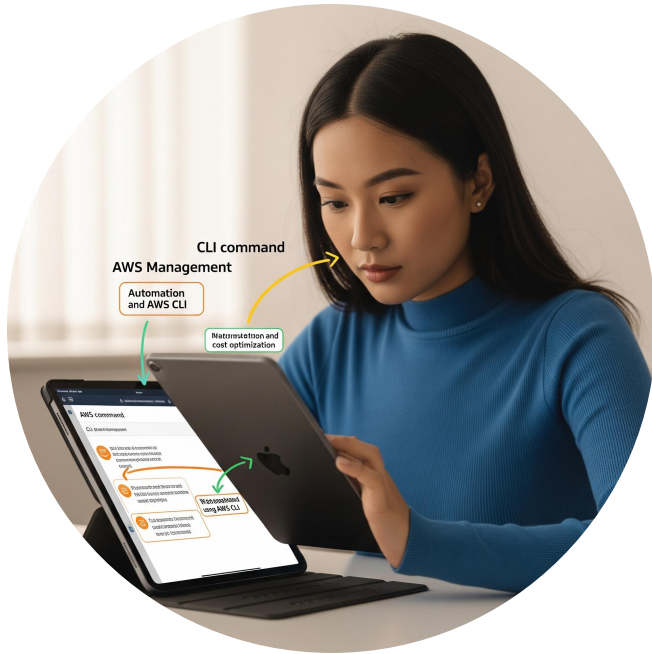
- a) PowerShell cmdlets are only available for Windows environments.
- b) Cmdlets provide a more user-friendly and object-oriented interface.
- c) Direct API calls are always faster than using cmdlets.
- d) PowerShell cmdlets do not require the AWS Tools for PowerShell.

Knowledge Check-6

Which Python library is commonly used for interacting with AWS services?

- a) requests
- b) pandas
- c) boto3
- d) numpy

Key Takeaways



- Review all topics discussed for main ideas.
- Summarize core concepts concisely for impact.
- Highlight the practical applications of CLI.
- Emphasize benefits for AWS management.